

DATA HANDLING AND VISUALIZATION (DSA06)

EXPERIMENTS 1 TO 12 REPORT (PYTHON VERSION)

Experiment 1: Monthly Sales Line Chart

Question / Task:

Create a line chart to visualize the monthly sales. Label the axes and title the chart appropriately.

Python Program Code:

```
# Experiment 1: Monthly Sales Line Chart (Python)
import matplotlib.pyplot as plt
import pandas as pd

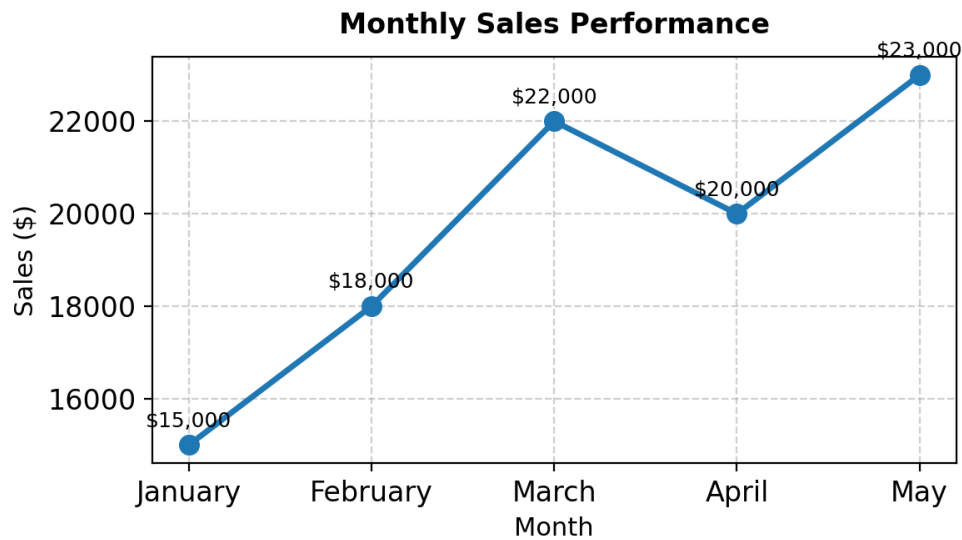
month = ["January", "February", "March", "April", "May"]
sales = [15000, 18000, 22000, 20000, 23000]
df = pd.DataFrame({"month": month, "sales": sales})

plt.figure(figsize=(6, 3.5))
plt.plot(df["month"], df["sales"], marker='o', color='#1f77b4', linewidth=2.5)
plt.title("Monthly Sales Performance")
plt.xlabel("Month")
plt.ylabel("Sales (in $)")
plt.grid(True, linestyle='--', alpha=0.6)

for i, txt in enumerate(sales):
    plt.annotate(f"${txt:,}", (month[i], sales[i] + 400), ha='center', fontsize=9)

plt.tight_layout()
plt.show()
```

Output Screenshot:



Experiment 2: Top-Selling Products Bar Chart

Question / Task:

Generate a bar chart to display the top-selling products for the year. Label the chart elements.

Python Program Code:

```
# Experiment 2: Top-Selling Products Bar Chart (Python)
import matplotlib.pyplot as plt

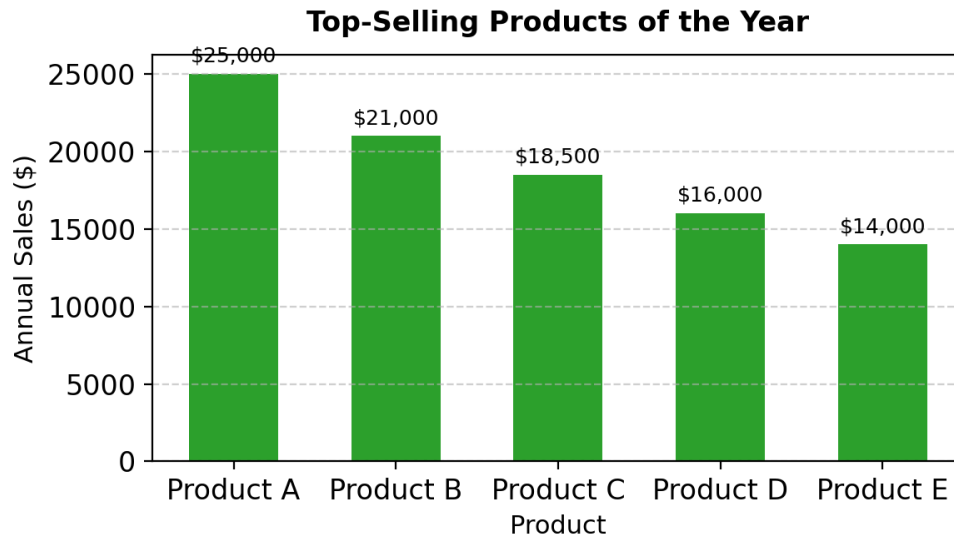
products = ["Product A", "Product B", "Product C", "Product D", "Product E"]
sales = [25000, 21000, 18500, 16000, 14000]

plt.figure(figsize=(6, 3.5))
bars = plt.bar(products, sales, color='#2ca02c', width=0.55)
plt.title("Top-Selling Products of the Year")
plt.xlabel("Product")
plt.ylabel("Annual Sales ($)")
plt.ylim(0, 30000)
plt.grid(axis='y', linestyle='--', alpha=0.6)

for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 500, f"${yval:,}", ha='center',
va='bottom', fontsize=9)

plt.tight_layout()
plt.show()
```

Output Screenshot:



Experiment 3: Advertising Budget vs Monthly Sales Scatter Plot

Question / Task:

Develop a scatter plot to explore the relationship between advertising budget and monthly sales. Explain the insights drawn from the scatter plot.

Python Program Code:

```
# Experiment 3: Scatter Plot & Trend Analysis (Python)
import matplotlib.pyplot as plt
import numpy as np

budget = [2000, 2500, 3200, 3000, 3500]
sales = [15000, 18000, 22000, 20000, 23000]

plt.figure(figsize=(6, 3.5))
plt.scatter(budget, sales, color='#d62728', s=80, edgecolors='black', zorder=3)

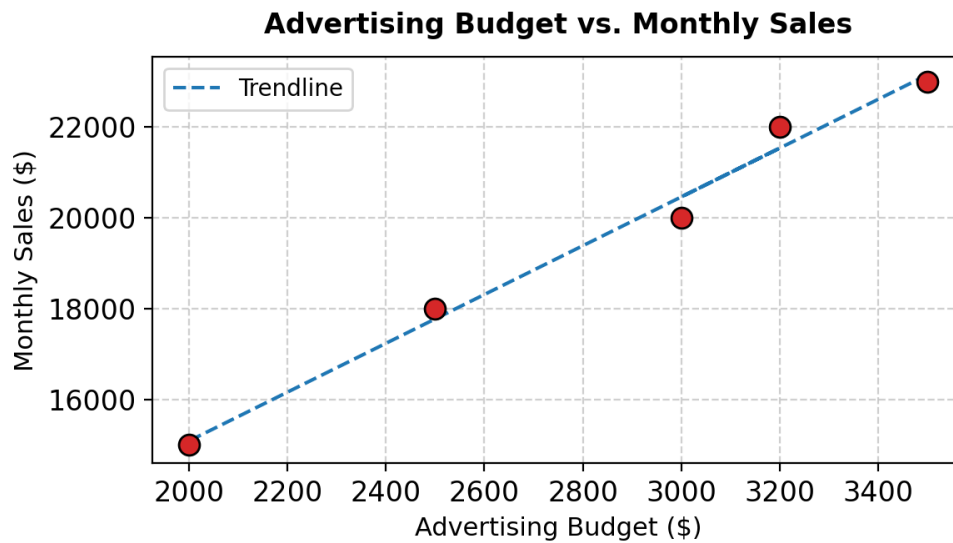
# Trendline
m, b = np.polyfit(budget, sales, 1)
plt.plot(np.array(budget), m * np.array(budget) + b, color='#1f77b4', linestyle='--',
label='Trendline')

plt.title("Advertising Budget vs. Monthly Sales")
plt.xlabel("Advertising Budget ($)")
plt.ylabel("Monthly Sales ($)")
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()

print("Insight: A strong positive linear correlation exists between advertising budget
and monthly sales.")
```

```
plt.tight_layout()
plt.show()
```

Output Screenshot:



Experiment 4: Interactive Sales Dashboard Layout

Question / Task:

Build an interactive dashboard combining the line chart and bar chart to allow users to explore sales data interactively.

Python Program Code:

```
# Experiment 4: Combined Dashboard View (Python)
import matplotlib.pyplot as plt

month = ["Jan", "Feb", "Mar", "Apr", "May"]
sales = [15000, 18000, 22000, 20000, 23000]
products = ["Prod A", "Prod B", "Prod C", "Prod D"]
p_sales = [25000, 21000, 18500, 16000]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(8, 3.5))

# Subplot 1: Line Chart
ax1.plot(month, sales, marker='s', color='#1f77b4', linewidth=2)
ax1.set_title("Sales Trend")
ax1.set_xlabel("Month")
ax1.set_ylabel("Sales ($)")
ax1.grid(True, linestyle='--', alpha=0.5)

# Subplot 2: Bar Chart
ax2.bar(products, p_sales, color='#ff7f0e', width=0.5)
ax2.set_title("Product Performance")
```

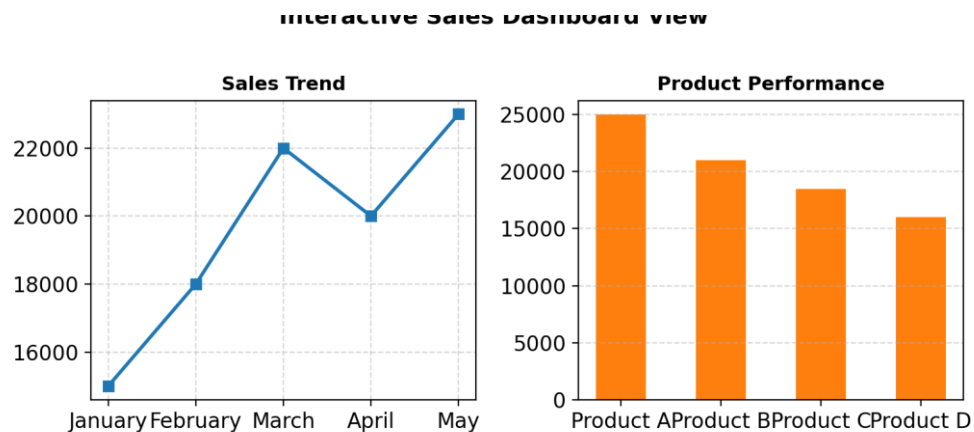
```

ax2.set_xlabel("Product")
ax2.set_ylabel("Sales ($)")
ax2.grid(axis='y', linestyle='--', alpha=0.5)

plt.suptitle("Interactive Sales Dashboard View", fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

```

Output Screenshot:



Experiment 5: Customer Age Distribution Histogram

Question / Task:

Create a histogram to represent the distribution of customer ages. Label the axes and title the chart.

Python Program Code:

```

# Experiment 5: Customer Age Histogram (Python)
import matplotlib.pyplot as plt

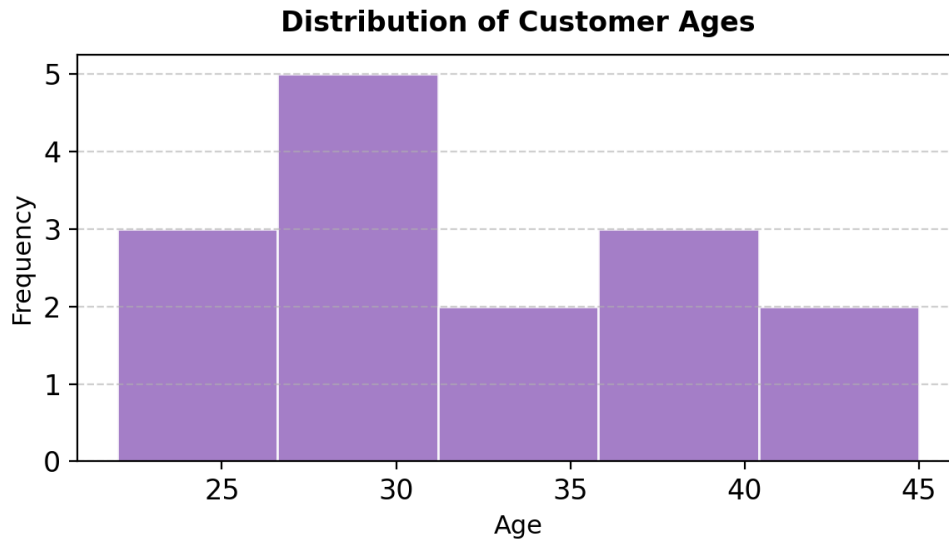
ages = [25, 30, 35, 28, 40, 22, 31, 29, 38, 45, 27, 33, 36, 41, 24]

plt.figure(figsize=(6, 3.5))
plt.hist(ages, bins=5, color='#9467bd', edgecolor='white', alpha=0.85)
plt.title("Distribution of Customer Ages")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.grid(axis='y', linestyle='--', alpha=0.6)

plt.tight_layout()
plt.show()

```

Output Screenshot:



Experiment 6: Customer Satisfaction Score Pie Chart

Question / Task:

Generate a pie chart to display the overall distribution of customer satisfaction scores. Include labels.

Python Program Code:

```
# Experiment 6: Customer Satisfaction Pie Chart (Python)
import matplotlib.pyplot as plt

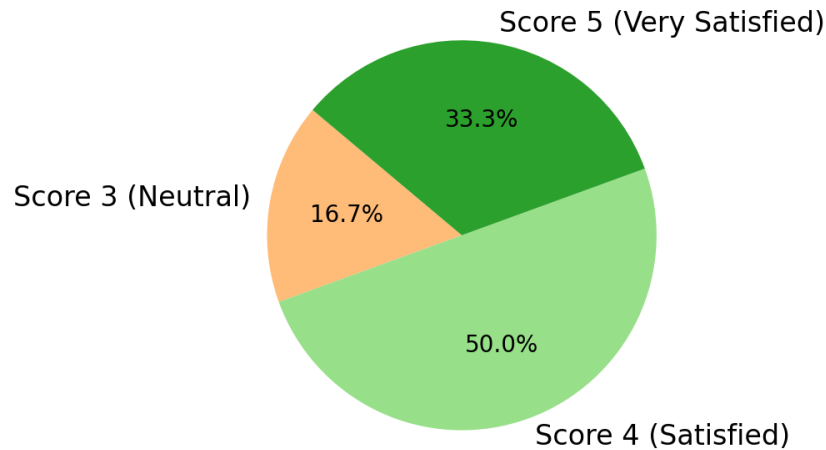
scores_count = [1, 3, 2]
labels = ["Score 3 (Neutral)", "Score 4 (Satisfied)", "Score 5 (Very Satisfied)"]
colors = ['#ffbb78', '#98df8a', '#2ca02c']

plt.figure(figsize=(6, 3.5))
plt.pie(scores_count, labels=labels, autopct='%1.1f%%', startangle=140, colors=colors)
plt.title("Customer Satisfaction Score Distribution")

plt.tight_layout()
plt.show()
```

Output Screenshot:

Customer Satisfaction Score Distribution



Experiment 7: Satisfaction Score by Age Group Stacked Bar Chart

Question / Task:

Build a stacked bar chart to visualize the distribution of customer satisfaction scores by age group.

Python Program Code:

```
# Experiment 7: Stacked Bar Chart (Python)
import matplotlib.pyplot as plt

age_groups = ['18-29', '30-39', '40+']
score_3 = [0, 1, 0]
score_4 = [2, 0, 0]
score_5 = [0, 1, 1]

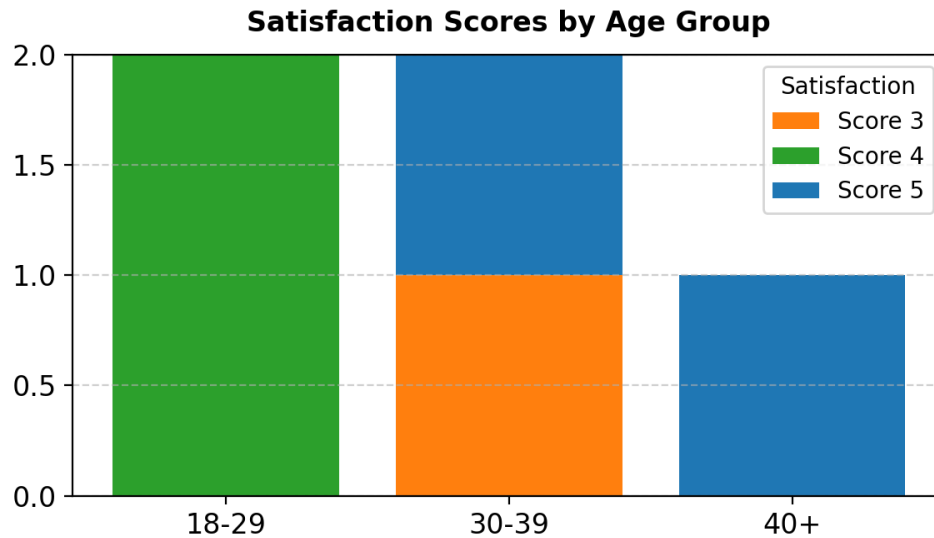
plt.figure(figsize=(6, 3.5))
plt.bar(age_groups, score_3, label='Score 3', color='#ff7f0e')
plt.bar(age_groups, score_4, bottom=score_3, label='Score 4', color='#2ca02c')

bottom_4 = [s3 + s4 for s3, s4 in zip(score_3, score_4)]
plt.bar(age_groups, score_5, bottom=bottom_4, label='Score 5', color='#1f77b4')

plt.title("Satisfaction Scores by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Count")
plt.legend(title='Satisfaction')
plt.grid(axis='y', linestyle='--', alpha=0.6)

plt.tight_layout()
plt.show()
```

Output Screenshot:



Experiment 8: Customer Feedback Word Cloud

Question / Task:

Develop a word cloud from open-ended customer feedback to identify prevalent customer sentiments.

Python Program Code:

```
# Experiment 8: Feedback Sentiment Word Cloud (Python)
from wordcloud import WordCloud
import matplotlib.pyplot as plt

text_data = {
    "Excellent": 50, "Good Service": 40, "Fast Support": 35,
    "User Friendly": 30, "Reliable": 28, "Needs Improvement": 15,
    "Great Interface": 25, "Helpful": 20
}

wc = WordCloud(width=600, height=350,
background_color='white').generate_from_frequencies(text_data)

plt.figure(figsize=(6, 3.5))
plt.imshow(wc, interpolation='bilinear')
plt.axis('off')
plt.title("Customer Feedback Word Cloud")

plt.tight_layout()
plt.show()
```

Output Screenshot:

Customer Feedback Word Cloud



Experiment 9: Employee Performance Trend Line Chart

Question / Task:

Create a line chart to visualize the performance trend of employees over time. Include a legend and labels.

Python Program Code:

```
# Experiment 9: Employee Performance Line Chart (Python)
import matplotlib.pyplot as plt

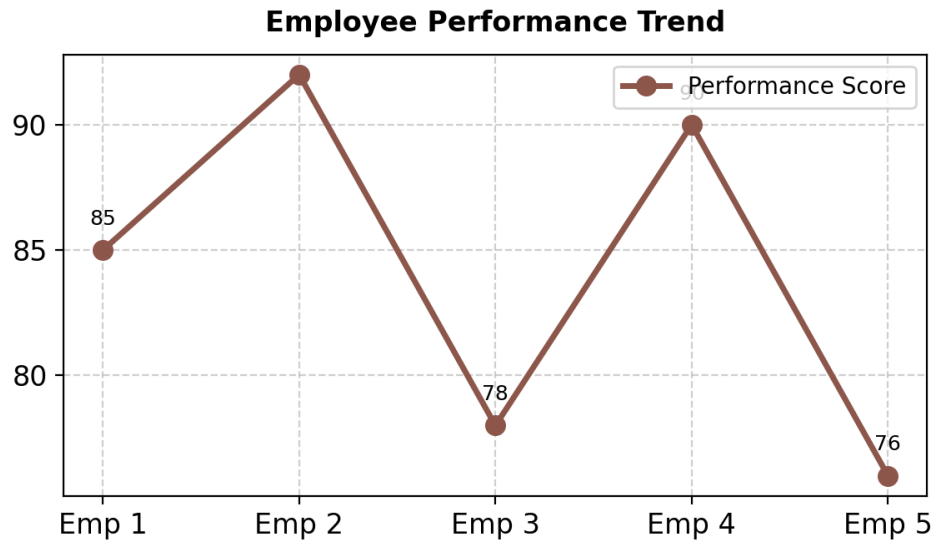
emp_ids = ["Emp 1", "Emp 2", "Emp 3", "Emp 4", "Emp 5"]
scores = [85, 92, 78, 90, 76]

plt.figure(figsize=(6, 3.5))
plt.plot(emp_ids, scores, marker='o', color='#8c564b', linewidth=2.5, label='Performance Score')
plt.title("Employee Performance Trend")
plt.xlabel("Employee ID")
plt.ylabel("Performance Score")
plt.ylim(70, 100)
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()

for i, txt in enumerate(scores):
    plt.annotate(str(txt), (emp_ids[i], scores[i] + 1), ha='center', fontsize=9)

plt.tight_layout()
plt.show()
```

Output Screenshot:



Experiment 10: Department Distribution Bar Chart

Question / Task:

Generate a bar chart showing the distribution of employees across different departments. Label the chart elements.

Python Program Code:

```
# Experiment 10: Department Distribution Bar Chart (Python)
import matplotlib.pyplot as plt
import pandas as pd

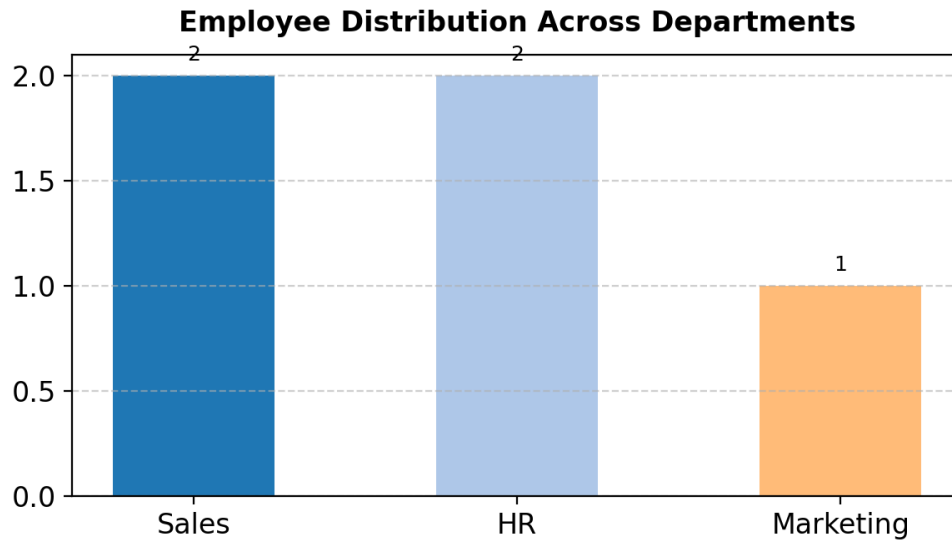
depts_data = ["Sales", "HR", "Marketing", "Sales", "HR"]
counts = pd.Series(depts_data).value_counts()

plt.figure(figsize=(6, 3.5))
bars = plt.bar(counts.index, counts.values, color=['#1f77b4', '#aec7e8', '#ffbb78'],
               width=0.5)
plt.title("Employee Distribution Across Departments")
plt.xlabel("Department")
plt.ylabel("Number of Employees")
plt.ylim(0, 3)
plt.grid(axis='y', linestyle='--', alpha=0.6)

for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.05, str(int(yval)), ha='center',
             va='bottom', fontsize=9)

plt.tight_layout()
plt.show()
```

Output Screenshot:



Experiment 11: Years of Service vs Performance Score Scatter Plot

Question / Task:

Build a scatter plot to analyze the correlation between years of service and performance scores. Explain any insights.

Python Program Code:

```
# Experiment 11: Service vs Performance Scatter Plot (Python)
import matplotlib.pyplot as plt

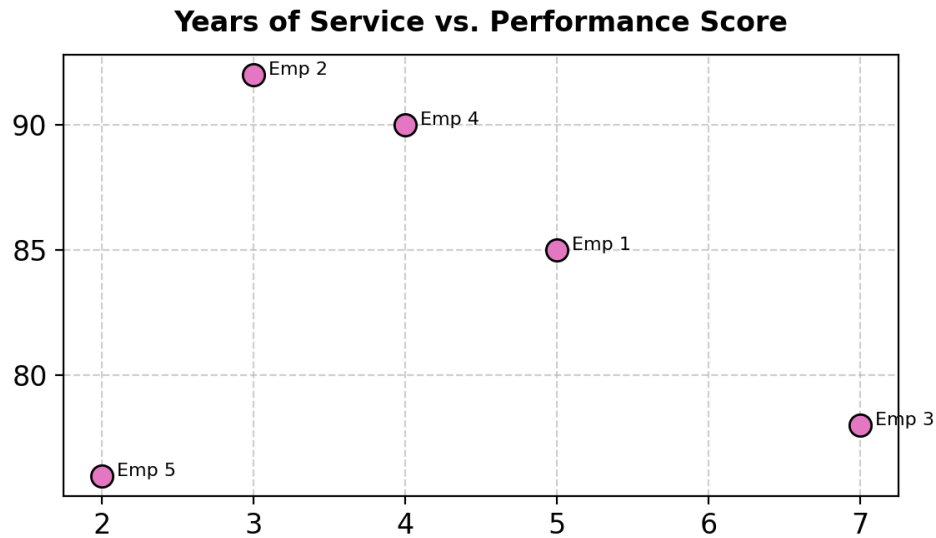
service = [5, 3, 7, 4, 2]
scores = [85, 92, 78, 90, 76]

plt.figure(figsize=(6, 3.5))
plt.scatter(service, scores, color='#e377c2', s=90, edgecolors='black', zorder=3)
plt.title("Years of Service vs. Performance Score")
plt.xlabel("Years of Service")
plt.ylabel("Performance Score")
plt.grid(True, linestyle='--', alpha=0.6)

for i in range(len(service)):
    plt.annotate(f"Emp {i+1}", (service[i] + 0.1, scores[i]), fontsize=8)

plt.tight_layout()
plt.show()
```

Output Screenshot:



Experiment 12: Interactive Tableau Dashboard View

Question / Task:

Develop a Tableau dashboard with the line chart and bar chart for interactive exploration.

Python Program Code:

```
# Experiment 12: Combined Dashboard Simulation (Python)
import matplotlib.pyplot as plt

emp_ids = ["Emp 1", "Emp 2", "Emp 3", "Emp 4", "Emp 5"]
scores = [85, 92, 78, 90, 76]
depts = ["Sales", "HR", "Marketing"]
counts = [2, 2, 1]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(8, 3.5))

# Subplot 1: Trend Line Plot
ax1.plot(emp_ids, scores, marker='o', color='#8c564b', linewidth=2)
ax1.set_title("Performance Trend")
ax1.set_xlabel("Employee ID")
ax1.set_ylabel("Score")
ax1.grid(True, linestyle='--', alpha=0.5)

# Subplot 2: Department Count Bar Plot
ax2.bar(depts, counts, color='#1f74b4', width=0.5)
ax2.set_title("Department Headcount")
ax2.set_xlabel("Department")
ax2.set_ylabel("Count")
ax2.grid(axis='y', linestyle='--', alpha=0.5)

plt.suptitle("Tableau Dashboard: Employee Performance Analysis", fontsize=13,
```

```
fontweight='bold', y=1.02)  
plt.tight_layout()  
plt.show()
```

Output Screenshot:

Tableau Dashboard: Employee Performance Analysis

